



Le génie pour l'industrie
Département de génie logiciel et des TI

 | **Département d'informatique**

La gestion plus moderne des projets logiciels – chapitre 3

Professeur Alain April, ing. PHD
(avec la contribution de Sylvie Trudel, PhD, UQAM)



Le contenu de ce document est protégé par le droit d'auteur. Ce document peut être utilisé à des fins d'étude personnelle seulement. Aucune partie de cette œuvre ne peut être transmise ou reproduite d'une quelconque façon sans la permission du détenteur du droit d'auteur.



1

Chapitre 3: La gestion de projet classique d'un projet logiciel

- 3.4 Le modèle de cycle de vie Kanban
- 3.5 Coordonner plusieurs équipes agiles d'un même projet
- 3.6 L'approche DevOps

2

2

3.4 - Kanban

- Kanban:
 - Le découpage des tâches d’une manière égale (ex. 2 à 3 jours);
 - Une limite de tâches par colonne
 - Travaux en cours, de l’anglais « WIP » (Work In Progress);
 - Des sous-colonnes avec la notion de « en cours » et « terminé »
 - Des règles pour dire qu’une tâche est vraiment terminée;
 - Pas d’échéance d’itérations, pas de réunions de planification, de fin d’étape...;
- Kanban = Itératif
- On mesure le « temps de cycle », soit la durée entre le moment où on commence à travailler sur une story et le moment où elle est « terminée » → On cherche à l’optimiser

3

3

3.4 - Kanban

Pool of Ideas	Feature Preparation		Feature Selected	User Story Identified	User Story Preparation		User Story Development		Feature Acceptance		Deployment	Delivered
	3 - 10		2 - 5	30	15		15		8		5	
	In Progress	Ready			In Progress	Ready	In Progress	Ready (Done)	In Progress	Ready		
Epic 431												Epic 294
Epic 478	Epic 444	Epic 662	Epic 602			Story 400-04	Story 401-04	Story 402-04	Epic 401	Epic 609	Epic 694	Epic 386
Epic 562	Epic 589		Epic 302	Story 300-04	Story 301-04	Story 302-04	Story 303-04	Story 304-04	Epic 468	Epic 577	Epic 276	Epic 419
Epic 439	Epic 651		Epic 335	Story 330-04	Story 331-04	Story 332-04	Story 333-04	Story 334-04	Epic 362		Epic 339	Epic 388
Epic 329				Story 320-04	Story 321-04	Story 322-04	Story 323-04	Story 324-04			Epic 521	Epic 287
Epic 287			Epic 512	Story 510-04	Story 511-04	Story 512-04	Story 513-04				Epic 582	Epic 274
Epic 606	Discarded											
	Epic 511	Epic 213										
	Epic 221											

19 janvier 2025

4

4

3.4 - Kanban

- Kanban:
 - Pourquoi et comment établir les limites par colonnes:
 - Identifier le nombre de développeurs +30%
 - Mettez la limite max de #dev X 2 = «stories » prêtes pour développer
 - Mettre les entrées (entrées: features et sorties: déploiement) à dev/3
 - Il faut éviter d’avoir trop de chats à fouetter:
 - Gère mieux la capacité (personnes 100% utilisées)
 - Nous force à classer logiquement le travail pour livrer tous les jours en production
 - Démarrer seulement ce qu’on peut finir en 2-3 jours
 - Prévient l’accumulation de travail débuté, mais pas terminé
 - Élimine de changer de sujet d’une « story » à une autre tout le temps
 - Élimine les meetings pour estimer p.c.q. la story va sortir en 2-3 jours
 - Facilite la communication

5

5

3.4 - Kanban

Pool of Ideas	Feature Preparation		Feature Selected	User Story Identified	User Story Preparation		User Story Development		Feature Acceptance		Deploy-ment	Delivered
	3 - 10		2 - 5	30	15		15		8		5	
	In Progress	Ready			In Progress	Ready	In Progress	Ready (Done)	In Progress	Ready		
Epic 431												Epic 294
Epic 478	Epic 444	Epic 662	Epic 602						Epic 401	Epic 609	Epic 694	Epic 386
Epic 562	Epic 589		Epic 302						Epic 468	Epic 577	Epic 276	Epic 419
Epic 439									Epic 362		Epic 339	Epic 388
Epic 329	Epic 651		Epic 335								Epic 521	Epic 287
Epic 287											Epic 582	Epic 274
Epic 606	Discarded		Epic 512									
	Epic 511	Epic 213										
	Epic 221											

19 janvier 2025

6

6

3.4 - Kanban

- Kanban:
 - Le découpage des tâches d'une manière égale (2 à 3 jours): Cette approche évite les réunions de planification et l'estimation, car tout est brisé en petits morceaux
 - Apprenez à découper en tâches de 2-3 jours:
 - Par élément fonctionnel (exemple une recherche par nombreux paramètres)
 - Par fonctionnalité CRUD
 - Par les étapes ou les rôles d'une transaction (enregistrement, logon, ..)
 - Par cas de test
 - Par difficulté d'implantation

7

7

3.4 - Kanban

- Kanban:
 - Les stand-up meetings diffèrent un peu:
 - Qu'est-ce qui est le plus près d'être fini et livrable?
 - Que pouvons-nous faire aujourd'hui pour l'enlever du tableau?
 - Quelqu'un travaille-t-il sur quelque chose qui ne figure pas au tableau?
 - Est-ce que quelque chose bloque notre progrès?
 - Quelqu'un est-il disponible pour aider à déplacer la carte X vers «Terminée»?

8

8

3.5 – Agile à grande échelle

- Il y a plusieurs méthodes pour coordonner le travail d'un ensemble d'équipes Agiles ensemble:
 - Les stand-up meetings diffèrent un peu:
 - Instaurer la livraison continue de toutes les équipes
 - Créer un Backlog commun à toutes les équipes
 - Favoriser une culture collaborative
 - Considérer l'utilisation d'une méthode à grande échelle:
 - Scaled Agile Framework de Dean Leffingwell (SAFe)
 - Disciplined Agile Delivery de Scott Ambler (DAD)
 - Large Scale Scrum de Craig Larman (LeSS)
 - Scrum of Scrums (SoS)
 - LeadingAgile

9

9

3.5 – SAFe

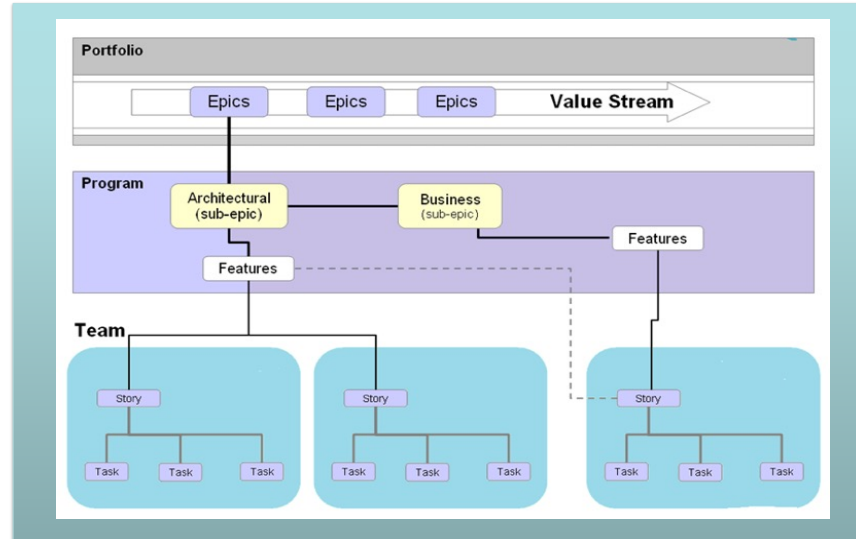
SAFe opère jusqu'à quatre niveaux: équipe, programme, solution [d'affaires] et portfolio:

- Cycles de livraison de 5 Sprints appelés « incréments de programme » (PI – Program Increment):
 - 4 Sprints de livraison + 1 Sprint d'innovation et planning
- Démarrage d'un IP précédé du « PI Planning », soit l'activité clé de SAFe
 - Si on ne fait pas de « PI Planning », on ne fait pas de SAFe!
- Les livrables sont liés à des *features*
- Notion de Train de livraison (Release Train);
- Réunion bihebdomadaire des SCRUM Masters et Chef(s) de Produit

10

10

3.5 - SAFe

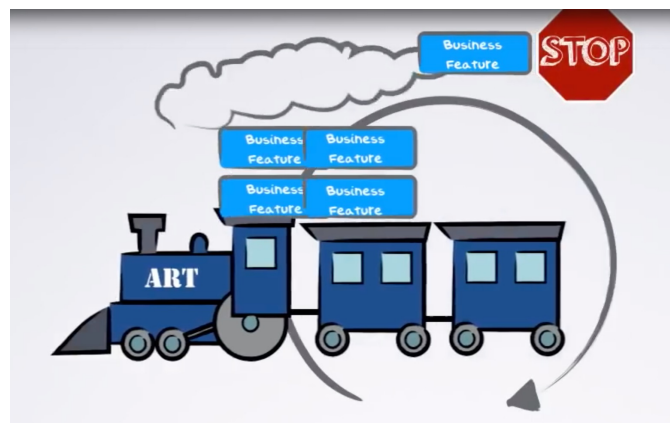


19 janvier 2025

11

11

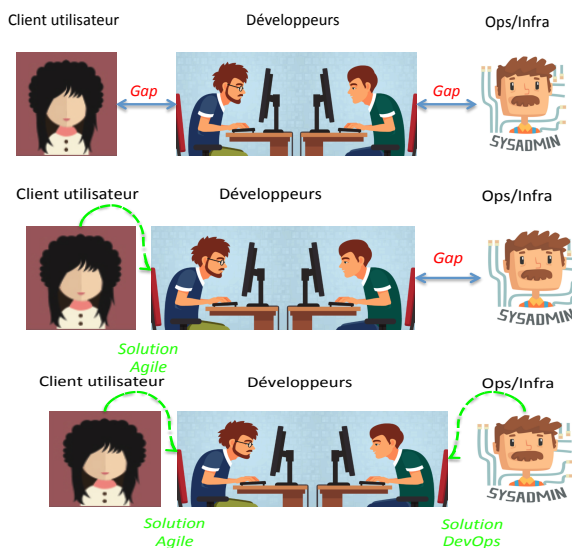
3.5 – SAFe – Agile Release Train



12

12

3.6 – L’approche DevOps



13

13

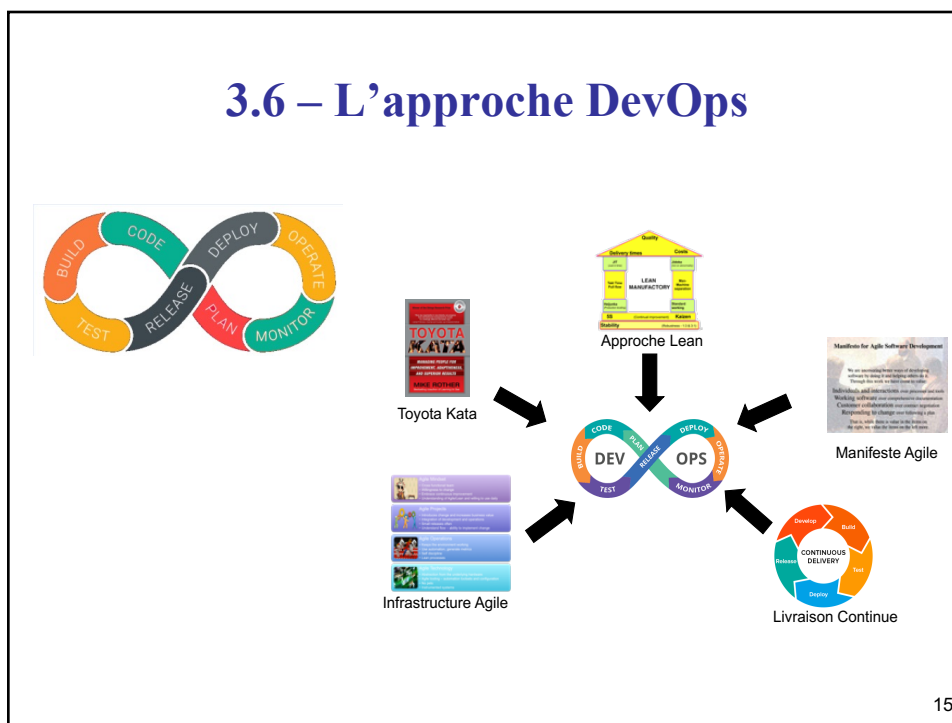
3.6 – L’approche DevOps

Agile	DevOps
Est intéressé par la livraison rapide et progressive de nouvelles fonctionnalités à valeur ajoutée	Est intéressé par les aspects de déploiement et d'opération du logiciel
Élimine l'écart entre les clients utilisateurs et l'équipe de développement pour accélérer les rétroactions concernant les exigences	Élimine l'écart entre l'équipe de développement et l'équipe d'infrastructure et opérations pour accélérer la transition vers la mise en production
Est centré sur les exigences fonctionnelles et non-fonctionnelles	Est centré sur la stabilité des opérations lors de mises en production répétées et s'assurer de l'état de préparation du logiciel prêt à être utilisé par ses utilisateurs
Met beaucoup d'accent sur la formation des membres d'équipe permettant, en cas de problème, que n'importe quel membre puisse aider à solutionner les problèmes	DevOps divise les tâches à des spécialistes mais s'assure d'une constante communication grâce à l'utilisation d'outils intégrés et à l'automatisation
Gère les livrables à l'aide de Sprints, soit la livraison de fonctionnalités à fréquence régulière (mois, 3 semaines, 2 semaines, ...)	DevOps vise à consolider ses livrables selon la valeur pour la clientèle, dès que disponible

14

14

3.6 – L'approche DevOps



15

3.6 – L'approche DevOps – Qu'est-ce que c'est?

5 principes clés:

- Qualité
- Petits lots
- Automatisation
- Amélioration continue
- Responsabilité

24 capacités
classées en 4 catégories:

- Techniques (11)
- Lean Management (4)
- Développement de produit Lean (4)
- Leadership transformationnel (5)

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

16

16

3.6 – L’approche DevOps – Les 11 capacités TECHNIQUES



Automatisation des tests



Automatisation du déploiement



Développement sur le tronc



Décalage à gauche de la sécurité



Architecture découplée



Équipes autonomes



Intégration Continue



Contrôle des versions



Gestion des données de test



Surveillance



Notification proactive

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

17

17

3.6 – L’approche DevOps – Les 4 capacités du Lean Management



Limite des travaux en cours



Gestion visuelle



Rétroaction de la production



Approbation simple des modifications

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

18

18

3.6 – L’approche DevOps – Les 4 capacités du Développement de produit Lean



Travailler en petits lots



Rendre le flux de travail visible



Recueillir et appliquer la rétroaction des clients



Expérimentation de l’équipe

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

19

19

3.6 – L’approche DevOps – Les 5 capacités du Leadership Transformationnel



Vision



Communication inspirante



Stimulation intellectuelle



Meneur supporteur



Reconnaissance personnelle

* Tiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

20

20

3.6 – L'approche DevOps – Mesurer la « performance de livraison de logiciel »

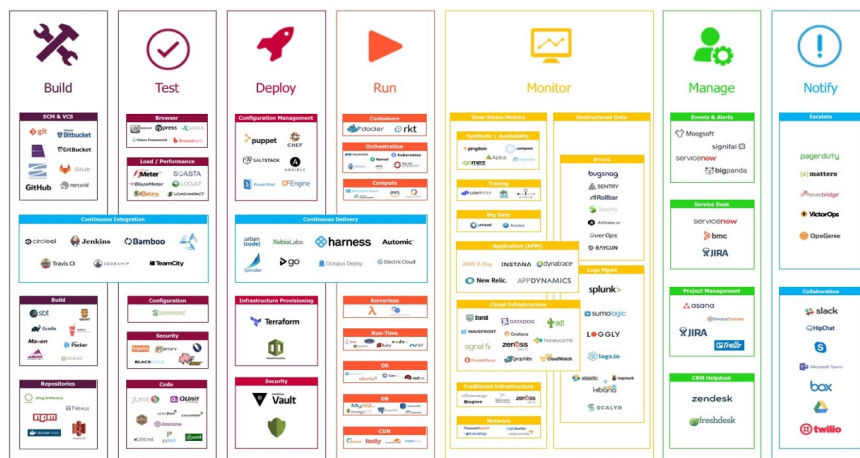
- 5 « mesures clés »:
 - Délai de réalisation → c'est le « temps de cycle » moyen
 - Fréquence de déploiement → en moyenne par développeur, pour pouvoir comparer les organisations entre elles, ou constater si on s'est amélioré malgré la variation du nombre de développeurs
 - Délai moyen de remise en service → suite à une panne
 - Pourcentage de défaillances → sur 100 changements, combien ont dû faire l'objet de corrections ou d'ajustements
 - Disponibilité → bien qu'on fasse souvent des MEP, le système doit demeurer disponible
- Pourquoi est-ce important?
 - Parce que la performance financière (ou non commerciale) de l'organisation est directement liée à sa performance de livraison de logiciel, et ce même si le développement logiciel n'est pas sa mission première (ex. une Banque, une cie d'assurance, une cie de transport, etc.)

* Inspiré de Nicole Forsgren PhD, Jez Humble et Gene Kim (2018). Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations, IT Revolution, ISBN: 978-1942788331.

21

21

3.6 – L'approche DevOps DevOps Lifecycle Mesh



22

22

En somme

Différentes méthodologies:

- **SCRUM**, itérations à durée fixe qui reviennent
- **Kanban**, board où les items passent d'une colonne à l'autre en fonction de leur avancée
- **SAFe**, gérer de grands projets (plusieurs équipes en parallèle en mode Agile/Kanban/DevOps)
- **DevOps**, ajout d'un rôle Ops dans l'équipe Dev Agile → Le DevOps, c'est de l'Agile++

23